Module-3

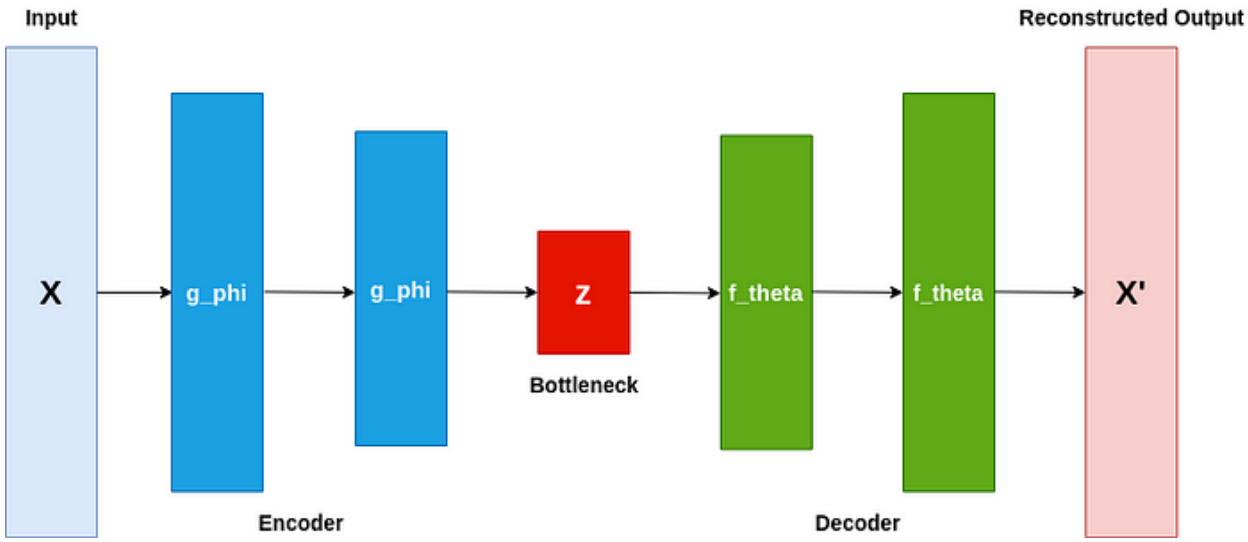
Autoencoders are neural network-based models that are used for unsupervised learning purposes to discover underlying correlations among data and represent data in a smaller dimension. The autoencoders frame unsupervised learning problems as supervised learning problems to train a neural network model. The input only is passed a the output. The input is squeezed down o a lower encoded representation using an encoder network, then a decoder network decodes the encoding to recreate back the input.

The encoding produced by the encoder layer has a lower-dimensional representation of the data and shows several interesting complex relationships among data.

An Autoencoder has the following parts:

1. **Encoder:** The encoder is the part of the network which takes in the input and produces a lower Dimensional encoding

2. **Bottleneck:** It is the lower dimensional hidden layer where the encoding is produced. The bottleneck layer has a lower number of nodes and the number of nodes in the bottleneck layer also gives the dimension of the encoding of the input.
3. **Decoder:** The decoder takes in the encoding and recreates back the input.

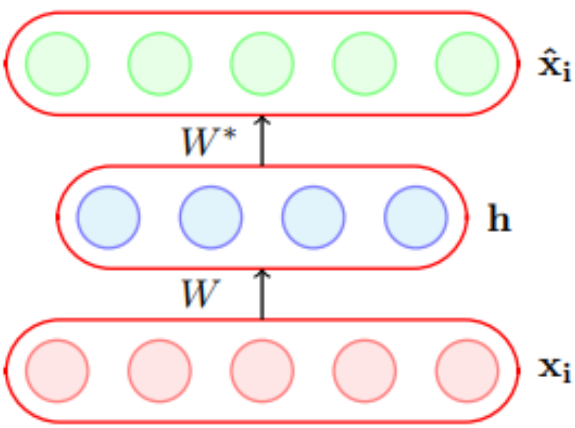


The bottleneck layer is the lower dimension layer. In the diagram, we have the neural networks encoder and decoder. Phi and Theta are the representing parameters of the encoder and decoder respectively.

The target of this model is such that the Input is equivalent to the Reconstructed Output. To achieve this we minimize a loss function

named **Reconstruction Loss.** Basically, Reconstruction Loss is given by the error between the input and the reconstructed output. It is usually given by the Mean Square error or Binary Crossentropy between the input and reconstructed output. Binary Crossentropy is used if the data is binary.

- Autoencoders can have recurrent connections and multiple hidden layers, but for now we will focus primarily on feedforward autoencoders with a single hidden layer, and thus on A(n, m, n) architectures.



$$\mathbf{h} = g(W\mathbf{x}_\mathbf{i} + \mathbf{b})$$
$$\hat{\mathbf{x}}_\mathbf{i} = f(W^*\mathbf{h} + \mathbf{c})$$

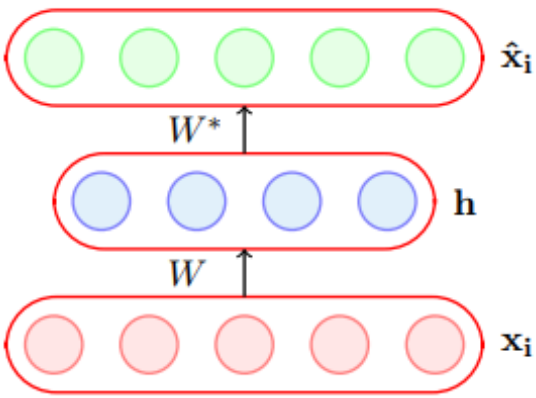
- An autoencoder is a special type of feed forward neural network which does the following
- Encodes its input $\mathbf{x}_\mathbf{i}$ into a hidden representation $\mathbf{h}$
- Decodes the input again from this hidden representation
- The model is trained to minimize a certain loss function which will ensure that $\hat{\mathbf{x}}_\mathbf{i}$ is close to $\mathbf{x}_\mathbf{i}$ (we will see some such loss functions soon)

Types of Autoencoder

- Undercomplete autoencoders
- Sparse autoencoders
- Contractive autoencoders
- Denoising autoencoders

Undercomplete Autoencoder

- An undercomplete autoencoder is a specific type of autoencoder where the dimensionality of the bottleneck layer (also known as the encoding layer or latent space) is smaller than the dimensionality of the input data.
- In other words, the undercomplete autoencoder is designed to learn a compressed representation of the input data that captures its most salient features while discarding some of the less important details.



$$\mathbf{h} = g(W\mathbf{x}_\mathbf{i} + \mathbf{b})$$
$$\hat{\mathbf{x}}_\mathbf{i} = f(W^*\mathbf{h} + \mathbf{c})$$

- Let us consider the case where $\dim(\mathbf{h}) < \dim(\mathbf{x}_\mathbf{i})$
- If we are still able to reconstruct $\hat{\mathbf{x}}_\mathbf{i}$ perfectly from $\mathbf{h}$, then what does it say about $\mathbf{h}$?
- $\mathbf{h}$ is a loss-free encoding of $\mathbf{x}_\mathbf{i}$. It captures all the important characteristics of $\mathbf{x}_\mathbf{i}$
- Do you see an analogy with PCA?

An autoencoder where $\dim(\mathbf{h}) < \dim(\mathbf{x}_\mathbf{i})$ is called an <u>under complete</u> autoencoder

- The concept of undercomplete autoencoders is closely related to the idea of dimensionality reduction.
- By forcing the autoencoder to learn a lower-dimensional representation of the input data, undercomplete autoencoders can help in tasks such as:

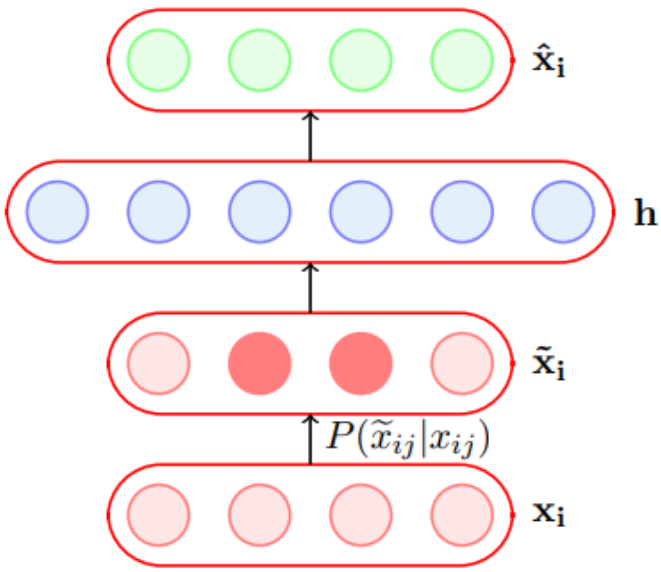
Feature Extraction: Undercomplete autoencoders can be used to automatically learn and extract important features from high-dimensional data, which can be useful for subsequent machine learning tasks.

Noise Reduction: The undercomplete bottleneck layer can act as a noise filter, preserving only the essential information from the input while ignoring noisy or irrelevant components.

Data Compression: Undercomplete autoencoders can compress the input data into a smaller representation, which can be useful for efficient storage, transmission, or processing of large datasets.

Denoising Autoencoders

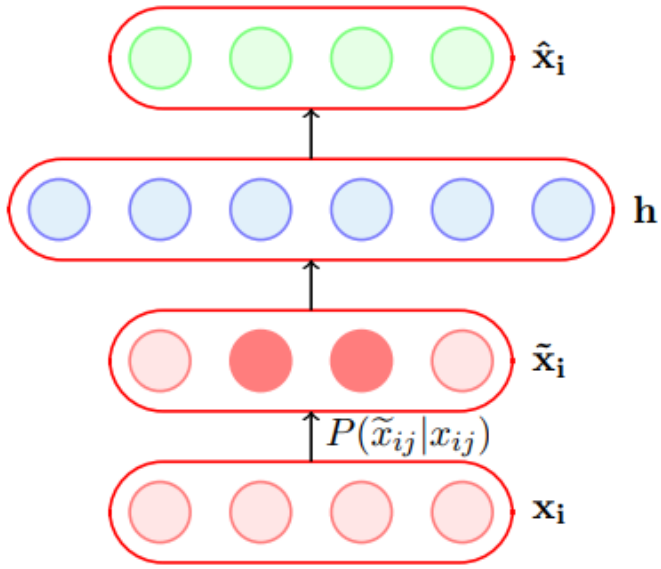
- Denoising autoencoders are trained to recover clean input data from noisy versions of the data.
- By forcing the model to reconstruct clean data from noisy samples, the autoencoder learns to capture the underlying structure of the data while ignoring noise.



- A denoising encoder simply corrupts the input data using a probabilistic process ($P(\widetilde{x}_{ij}|x_{ij})$) before feeding it to the network
- A simple $P(\widetilde{x}_{ij}|x_{ij})$ used in practice is the following

$$P(\widetilde{x}_{ij} = 0|x_{ij}) = q$$
$$P(\widetilde{x}_{ij} = x_{ij}|x_{ij}) = 1 - q$$

- In other words, with probability q the input is flipped to 0 and with probability $(1 - q)$ it is retained as it is

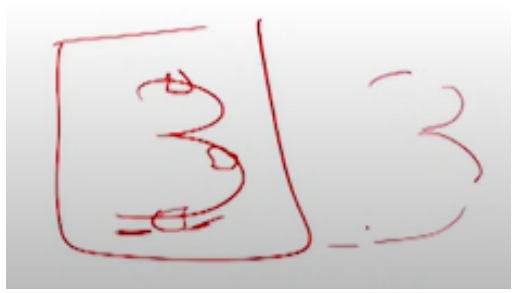


For example, it will have to learn to reconstruct a corrupted x_{ij} correctly by relying on its interactions with other elements of $\mathbf{x}_i$

- How does this help ?
- This helps because the objective is still to reconstruct the original (uncorrupted) $\mathbf{x}_i$

$$\arg\min_{\theta} \frac{1}{m} \sum_{i=1}^{m} \sum_{j=1}^{n} (\hat{x}_{ij} - x_{ij})^2$$

- It no longer makes sense for the model to copy the corrupted $\tilde{\mathbf{x}}_i$ into $h(\tilde{\mathbf{x}}_i)$ and then into $\hat{\mathbf{x}}_i$ (the objective function will not be minimized by doing so)
- Instead the model will now have to capture the characteristics of the data correctly.



- We saw one form of $P(\tilde{x}_{ij}|x_{ij})$ which flips a fraction q of the inputs to zero
- Another way of corrupting the inputs is to add a Gaussian noise to the input

$$\tilde{x}_{ij} = x_{ij} + \mathcal{N}(0, 1)$$

- Denoising autoencoders are a type of neural network architecture used in deep learning for **unsupervised feature learning and data denoising**.
- They are a variant of autoencoders, which are neural networks designed to learn efficient representations of input data by training to encode the data into a compact latent space and then decode it back to its original form.

- Denoising autoencoders extends this concept by adding a noise-corrupted version of the input data during training, forcing the autoencoder to learn to reconstruct the original, clean data from the noisy inputs.

How denoising autoencoders work:

Architecture:

- A denoising autoencoder consists of an encoder and a decoder, similar to a standard autoencoder.
- The encoder transforms the input data into a compressed representation in the latent space, while the decoder reconstructs the original data from this compressed representation.

Noise Injection:

- During training, noise is added to the input data before feeding it into the encoder.
- This noise can be random, such as Gaussian noise, or it can be artificially generated in various ways.
- The goal is to make the network learn to recover the clean data from the noisy version.

Training Objective:

- The network's objective during training is to minimize the difference between the reconstructed (decoded) data and the original, clean input data.
- This is typically done using a loss function such as mean squared error (MSE) or binary cross-entropy.

Regularization:

- Denoising autoencoders often exhibit better generalization capabilities compared to standard autoencoders.
- The added noise acts as a form of regularization, which can prevent overfitting and encourage the network to learn more robust and useful features.

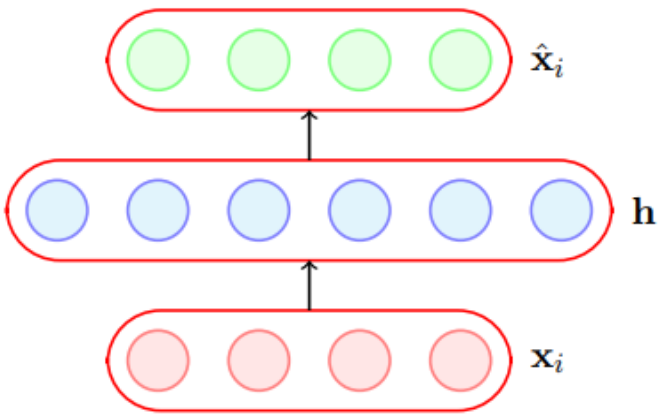
Advantages:

- Advantages of denoising autoencoders include their ability to learn meaningful representations of data, handle noisy or incomplete inputs, and reduce the risk of overfitting.

- They have been used in various applications, such as image denoising, data denoising and restoration, feature learning, and even as pretraining steps for downstream supervised tasks.

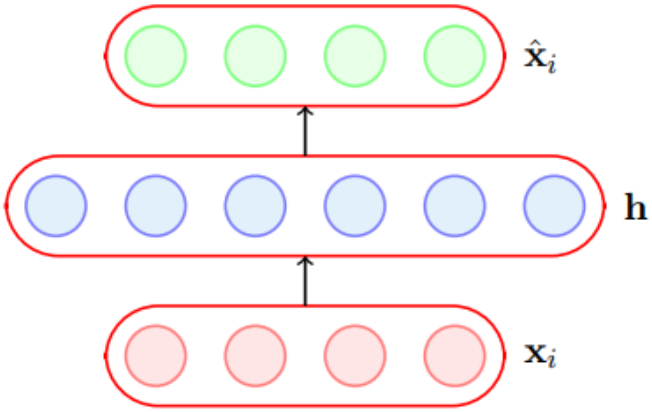
However, it's worth noting that while denoising autoencoders can be effective, they might require careful tuning of hyperparameters, such as the amount and type of noise added, the architecture of the encoder and decoder, and the choice of the loss function.

In summary, denoising autoencoders are a valuable tool in deep learning for learning robust representations of data and dealing with noisy inputs, ultimately contributing to improved performance in various machine learning tasks.



Sparse Autoencoders

- A hidden neuron with sigmoid activation will have values between 0 and 1
- We say that the neuron is activated when its output is close to 1 and not activated when its output is close to 0.
- A sparse autoencoder tries to ensure the neuron is inactive most of the times.



- If the neuron l is sparse (i.e. mostly inactive) then $\hat{\rho}_l \rightarrow 0$
- A sparse autoencoder uses a sparsity parameter ρ (typically very close to 0, say, 0.005) and tries to enforce the constraint $\hat{\rho}_l = \rho$
- One way of ensuring this is to add the following term to the objective function

The average value of the activation of a neuron l is given by

$$\hat{\rho}_l = \frac{1}{m} \sum_{i=1}^{m} h(\mathbf{x}_i)_l$$

$$\Omega(\theta) = \sum_{l=1}^{k} \rho \log \frac{\rho}{\hat{\rho}_l} + (1-\rho) \log \frac{1-\rho}{1-\hat{\rho}_l}$$

- The function will reach its minimum value(s) when $\hat{\rho}_l = \rho$.

- Now,

$$\hat{\mathscr{L}}(\theta) = \mathscr{L}(\theta) + \Omega(\theta)$$

- $\mathscr{L}(\theta)$ is the squared error loss or cross entropy loss and $\Omega(\theta)$ is the sparsity constraint.

- Sparse autoencoders are a type of neural network architecture used in deep learning for learning sparse representations of data.
- They are a variation of autoencoders, which are neural networks designed to learn efficient representations of input data by training to encode the data into a compressed latent space and then decode it back to its original form.
- Sparse autoencoders add a sparsity constraint to the learning process, encouraging the model to use only a limited number of neurons in the hidden layers.

Key components and concepts of sparse autoencoders in the context of deep learning are:

Architecture:

- Sparse autoencoders consist of an encoder and a decoder, like traditional autoencoders.
- The encoder transforms the input data into a sparse representation in the latent space, while the decoder reconstructs the original data from this sparse representation.

Sparsity Constraint:

- The sparsity constraint is added to the objective function during training. This constraint encourages the activation of only a subset of neurons in the hidden layer.
- This means that the autoencoder learns to activate only a small number of neurons at a time, leading to a sparse representation.

Sparsity Regularization:

- The sparsity constraint is usually enforced through regularization techniques.
- A common approach is to add a penalty term to the loss function that encourages sparsity.
- The penalty term can be based on metrics such as the average activation of neurons or the KL divergence between the desired sparsity and the actual activations.

Activation Functions:

- Certain activation functions, like the rectified linear unit (ReLU) and its variants, can promote sparsity by zeroing out negative activations.
- This encourages neurons to be active only when the input to the neuron is positive, leading to sparse representations.

Hyperparameters:

- Designing sparse autoencoders involves tuning hyperparameters related to the sparsity constraint, such as the weight of the sparsity penalty term and the desired level of sparsity.
- These hyperparameters play a crucial role in controlling the extent of sparsity and the quality of learned representations.

Sparse autoencoders have several potential benefits in deep learning:

Interpretability: Sparse representations are often more interpretable than dense ones. The activated neurons can correspond to specific features in the input data.

Data Compression: By using a sparse representation, the model can capture the most important features of the data while discarding less relevant information, leading to efficient data compression.

Robustness: Sparse autoencoders can learn to represent data in a more robust way, focusing on the most distinctive and invariant features.

- Sparse autoencoders have been used in various applications, such as image compression, feature learning, anomaly detection, and more.
- However, training sparse autoencoders can be challenging due to the complexity introduced by the sparsity constraint.
- Careful hyperparameter tuning and regularization techniques are important for successful training and effective use of sparse autoencoders in deep learning tasks.

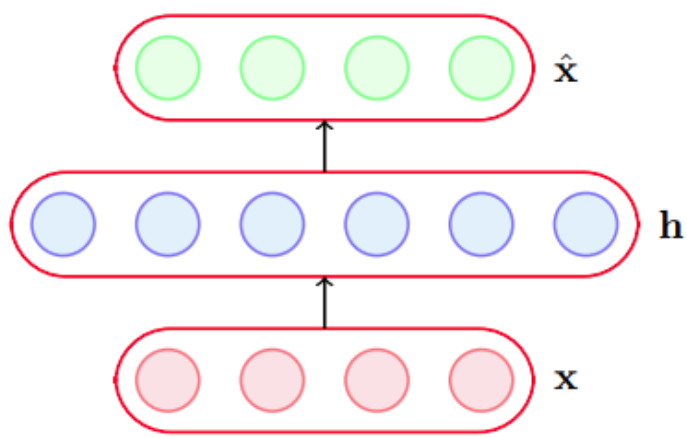
Contractive Autoencoders

- A contractive autoencoder also tries to prevent an overcomplete autoencoder from learning the identity function.
- It does so by adding the following regularization term to the loss function

$$\Omega(\theta) = \|J_{\mathbf{x}}(\mathbf{h})\|_F^2$$

where $J_{\mathbf{x}}(\mathbf{h})$ is the Jacobian of the encoder.

- Let us see what it looks like.



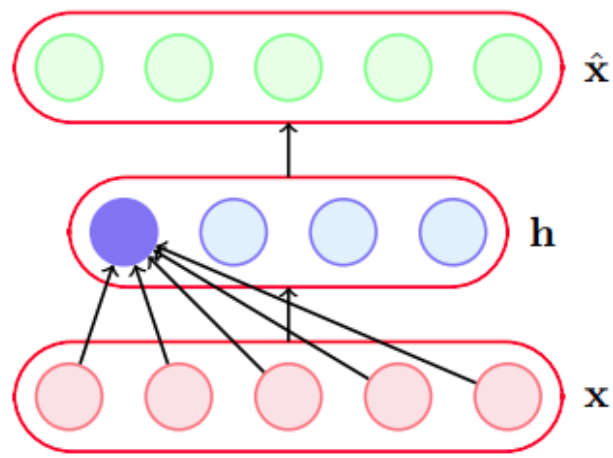
- If the input has n dimensions and the hidden layer has k dimensions then
- In other words, the (l, j) entry of the Jacobian captures the variation in the output of the lth neuron with a small variation in the jth input.

$$J_{\mathbf{x}}(\mathbf{h}) = \begin{bmatrix} \frac{\partial h_1}{\partial x_1} & \cdots & \cdots & \cdots & \frac{\partial h_1}{\partial x_n} \\ \frac{\partial h_2}{\partial x_1} & \cdots & \cdots & \cdots & \frac{\partial h_2}{\partial x_n} \\ \vdots & & \ddots & & \vdots \\ \frac{\partial h_k}{\partial x_1} & \cdots & \cdots & \cdots & \frac{\partial h_k}{\partial x_n} \end{bmatrix}$$

$$\|J_{\mathbf{x}}(\mathbf{h})\|_F^2 = \sum_{j=1}^{n} \sum_{l=1}^{k} \left(\frac{\partial h_l}{\partial x_j} \right)^2$$

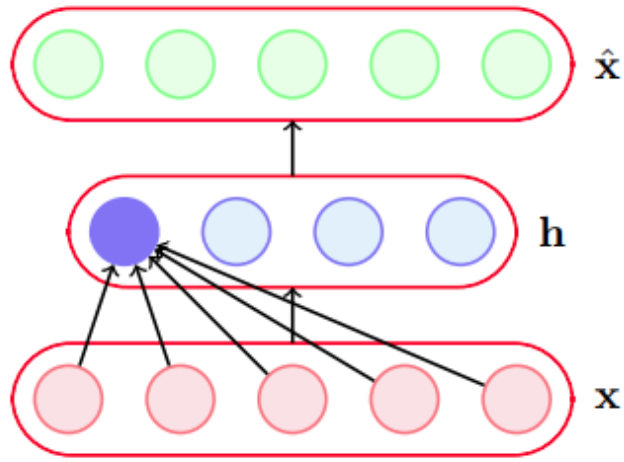
- What is the intuition behind this ?
- Consider $\frac{\partial h_1}{\partial x_1}$, what does it mean if $\frac{\partial h_1}{\partial x_1} = 0$
- It means that this neuron is not very sensitive to variations in the input x_1.
- But doesn't this contradict our other goal of minimizing $\mathcal{L}(\theta)$ which requires $\mathbf{h}$ to capture variations in the input.

$$\|J_{\mathbf{x}}(\mathbf{h})\|_F^2 = \sum_{j=1}^{n} \sum_{l=1}^{k} \left(\frac{\partial h_l}{\partial x_j} \right)^2$$

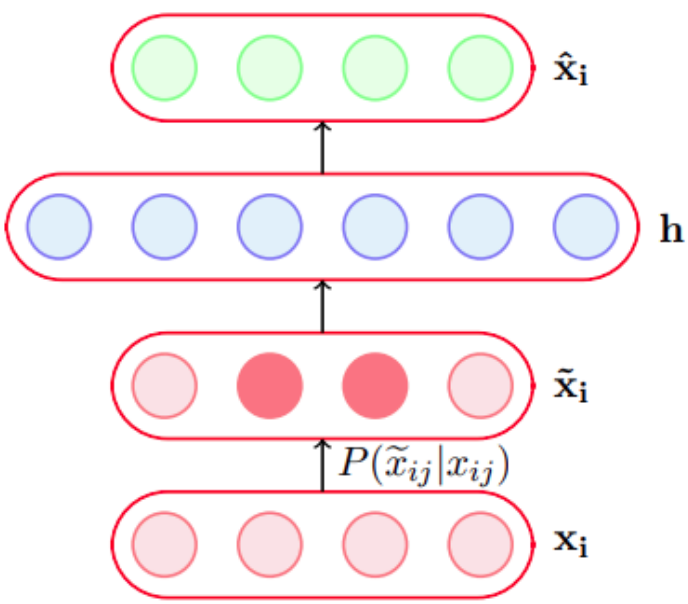


- Indeed it does and that's the idea
- By putting these two contradicting objectives against each other we ensure that $\mathbf{h}$ is sensitive to only very important variations as observed in the training data.
- $\mathcal{L}(\theta)$ - capture important variations in data
- $\Omega(\theta)$ - do not capture variations in data
- Tradeoff - capture only very important variations in the data

$$\|J_{\mathbf{x}}(\mathbf{h})\|_F^2 = \sum_{j=1}^{n}\sum_{l=1}^{k}\left(\frac{\partial h_l}{\partial x_j}\right)^2$$



Summary



Regularization

$\Omega(\theta) = \lambda\|\theta\|^2$ Weight decaying

$\Omega(\theta) = \sum_{l=1}^{k} \rho \log \frac{\rho}{\hat{\rho}_l} + (1-\rho)\log\frac{1-\rho}{1-\hat{\rho}_l}$ Sparse

$\Omega(\theta) = \sum_{j=1}^{n}\sum_{l=1}^{k}\left(\frac{\partial h_l}{\partial x_j}\right)^2$ Contractive